

METHONTOLOGY 把建本体从手艺变成工程

METHONTOLOGY: From Ontological Art Towards Ontological Engineering

本篇范围说明：这是一篇 1997 年的 AAI 研讨会论文（AAAI Technical Report SS-97-06，马德里理工大学）。下面「全文翻译」一节覆盖它的摘要、引言、本体开发过程、生命周期、以及 METHONTOLOGY 六个阶段的主体内容；那份化学品本体的具体术语清单、以及中间表示的逐张表格样例不逐条翻译。[这是一篇近三十年前的论文，为什么现在还值得读，写在「点评」那一节。](#)

核心内容

Q：这篇论文要解决什么问题？

A：论文开头那句话就是它的判断——[在此之前，建本体是一门手艺，不是一项工程](#)。大量本体被不同团队用不同方法建出来了，但[几乎没有人发表过「该怎么建」](#)：实践、设计判据、活动、方法、工具，都没有标准化。作者说得很直接：正是因为缺少标准化的活动、生命周期、系统化方法、以及一套定义清楚的设计判据，本体开发才成了手艺而不是工程。

Q：作者认为「从手艺变成工程」的分界线在哪？

A：原文给的判据很具体：[当存在一个从「需求定义」一直到「成品维护」的、被定义和标准化了的生命周期，以及驱动它的方法与技术时，手艺才会变成工程](#)。

注意这句话里的两端——不是「有方法就够了」，是[要有一条从需求一直贯到维护的完整链条](#)。

Q：建本体和建专家系统有什么本质区别？为什么专家系统的方法不能直接搬过来？

A：这是全篇最要紧的一处区分，而它决定了后面整套方法的形状。

专家系统的困境：知识工程师最头疼的事是[拿不到需求](#)。需求要说清系统将怎么表现，而专家通常没法具体完整地描述自己在这个领域里是怎么做的。所以专家系统一般用「演进式原型」增量地建——[每一轮成品的不足，就当成下一轮原型的规格说明](#)。

本体不是这样：本体被建出来是为了[随时随地被复用或共享](#)，而且独立于使用它的那个应用的行为和领域。所以本体构建者应当能够——至少部分地——[事先说清这个本体将要覆盖的那一大部分词汇](#)。

结论：这就是本体开发过程与知识库系统开发过程的主要差别，因此[用来建知识库系统的方法论不能完整地套用到建本体上](#)。

Q: METHONTOLOGY 的六个阶段分别产出什么？

A: 这张表回答一个问题：每个阶段的产物是什么、判据是什么。「产物」一列是可交付的文档或代码，不是抽象活动。

阶段	产物	关键判据或做法
规格说明	一份本体需求规格说明文档	必须包含三件事： 目的 （含预期用途、使用场景、最终用户）、 形式化程度、范围 （要表示的术语集合、它们的特征与粒度）
知识获取	术语表初稿与中间表示	它是一项 独立活动 ，但和其他活动重叠。 大部分获取和规格说明阶段同时做，随着开发推进而递减
概念化	一个由一组明确定义的交付物表达的概念模型	先建完整的 术语表 （含概念、实例、动词、属性），再把术语分成概念组与动词组，各自建分类树与图
集成	一份集成文档	逐个术语记下：概念模型里的名字、要从哪个本体取它的定义、那个定义在源本体里的名字与参数
实现	用形式语言编码的本体	开发环境至少要提供：词法与语法分析器、翻译器、编辑器、浏览器、检索器、 评估器 （检测不完整、不一致、冗余知识）、自动维护器
评估	技术判断	贯穿每个阶段，不是最后一道

Q: 规格说明文档要满足哪三条性质？

A: 因为**你永远无法证明一份本体规格说明文档是完备的**（任何时间任何地点都可能有人发现一个该收进来的新术语），所以论文改为要求三条可检查的性质：

- **简洁性**：每一个术语都是相关的，没有重复或不相关的术语
- **部分完备性**：与术语的覆盖面、「停在哪一层」这个问题、以及每个术语的粒度有关
- **一致性**：所有术语及其含义在这个领域里都讲得通

Q: 为什么推荐「中间开花」而不是自上而下或自下而上？

A: 论文引 Uschold 的论证：**中间开花法的主要好处是，它让你先identify出这个本体的主要概念**。在这些术语和它们的定义上达成一致之后，**只在必要时**再往上泛化或往下特化。

结果是：你用的那些术语更稳定，需要的返工与总体工作量更少。

背景补充 / 点评

先回答一个必须先回答的问题：一篇 1997 年的论文，为什么现在值得读。

论文里提到的具体技术基本都过时了——Ontolingua、CLASSIC、BACK、LOOM 这些实现语言现在几乎没人用，Cyc 也早已不是主流选择。**但它解决的那个问题今天原样存在，只是换了外衣。**

今天的形态是这样的：一个团队决定「用本体来管住大模型的抽取」，然后直接开始写 YAML，写着写着发现概念之间关系混乱、同一个东西两个名字、加一个新维度要改五个地方。**这就是论文说的「手艺」状态**——没有需求规格说明、没有术语表、没有集成文档、没有「停在哪一层」的判据。

这篇论文的价值不在技术，在它把「建本体」这件事拆成了六个有交付物的阶段。有交付物意味着可以检查、可以交接、可以判断做没做完。而「凭感觉写 YAML」这件事没有中间状态可检查。

再说这篇论文里我认为最值得单独拿出来的一处：本体与专家系统的那个区分。

它的逻辑链是这样的：**专家系统只服务一个应用，所以它可以「先做出来再看哪里不对」；本体要被多个应用复用，所以它必须「先说清覆盖什么再做」。**

这条推理对今天有直接后果。现在很多做法是「让大模型先抽一批，看看抽出什么，再决定 schema」——这正是专家系统那套演进式原型的思路。按这篇论文的判断，**这个思路用在本体上是错位的**：不是说迭代不对，而是**如果一开始不说清覆盖范围，后面每个消费方都会按自己的需要往里加东西，而本体的价值恰恰在于多方共用同一份定义。**

顺带说一处它提前预见到的事。论文要求实现阶段的开发环境必须提供「评估器，用于检测不完整、不一致和冗余的知识」。1997 年这是一个愿望；而今天这正是「加载期 fail-fast 校验」在做的事——引用完整性、键名白名单、概念声明校验。**这一条从愿望变成了标准做法，中间隔了近三十年。**

这篇论文的留白。

第一，它没有回答「停在哪一层」这个问题，只是把它列成了「部分完备性」的一部分。原文用了 the stopover problem 这个说法，但没给判据。而这恰恰是实践中最难的一步——一个概念要不要再往下拆一层，论文说不出该看什么。

第二，它假设本体是人建的。六个阶段里的知识获取靠专家访谈、文本分析、知识获取工具。**它没有考虑「本体的一部分由程序生成、或者本体要约束程序的写入」这种情形**——那是 2026 年的问题，1997 年不存在。

第三，评估那一节最薄。它说评估贯穿每个阶段、引用了作者自己另一篇框架论文，但在这篇里没有给出任何可执行的评估判据。

对我自己的启示。这篇是昨天那批用户画像调研里唯一一篇讲「怎么建」而不是「建成什么样」的，所以它的用法和其余九篇不同。

第一处，也是最直接的一处：那六个阶段可以直接当用户画像本体的工作清单。上次会上定了三层（学员画像、memory、会话历史）和三个卡点，但那些是**内容**层面的决定。这篇给的是**过程**层面的：先写需求规格说明（目的、形式化程度、范围），再建术语表，再概念化，再看能从现成本体（GUMO 那些）集成什么，然后实现，评估贯穿全程。

而按这个清单一对照，现在缺的是第一步。我们有一堆调研（GUMO 的三段式、25 个字段、Quipu 的双时态），但没有一份写下来的需求规格说明文档——目的、形式化程度、范围这三样都还在会议记录和 digest 里，不在一份文档里。

第二处，「本体不能像专家系统那样先做再改」这一条要认真对待。现在很自然的做法是「先让大模型抽一批画像出来，看看长什么样再定 schema」。按这篇论文，这条路对专家系统成立、对本体不成立——因为本体要被多方复用（提取要用、消费要用、审计要用），而这三方的需要不一样。如果先抽再定，最后定出来的会是「抽取方便的那个 schema」，而不是三方都能用的那个。

第三处，规格说明的三条性质里，「部分完备性」那条给了我一个具体的提醒。它说完备性无法被证明——任何时间任何地点都可能有人发现一个该收进来的新术语。所以不要追求「把所有画像维度列全」，而要保证已列的每一个都相关（简洁性）、覆盖面与粒度说得清（部分完备性）、彼此讲得通（一致性）。

这一条和 GUMO 那个「167 个基本维度 + 255 个兴趣类目」放在一起看很有意思：GUMO 列了一千多组，而这篇论文会说那不是完备性的证明，只是覆盖面的一个当时状态。格子（25 个字段）要固定，清单（能填什么）可以增长——这正是昨天弄清的那个区分。

文中金句

"the absences of standardized activities, life cycles, and systematic methodologies as well as a set of well-defined design criteria, techniques and tools make the ontologies development a craft rather than an engineering activity." 正是因为缺少标准化的活动、生命周期、系统化方法论，以及一套定义清楚的设计判据、技术和工具，本体开发才成了一门手艺，而不是一项工程活动。

"the art will became engineering when there exist a definition and standardization of a life cycle that goes from requirements definition to maintenance of the finished product." 当存在一条从需求定义一直到成品维护的、被定义和标准化了的生命周期时，手艺才会变成工程。

"Ontologies are built to be reused or shared anytime, anywhere, and independently of the behavior and domain of the application that uses them. ... Consequently, methodologies used to build KBSs can not completely be applied to build ontologies." 本体被建出来是为了随时随地被复用或共享，并且独立于使用它的那个应用的行为与领域。……因此，用来建知识库系统的方法论不能完整地套用到建本体上。

"Since you can not prove the total completeness of your ontology specification document ..." 既然你无法证明本体规格说明文档的完全完备性……

摘要

This paper does not pretend either to transform completely the ontological art in engineering or to enumerate exhaustively the complete set of works that has been reported in this area. Its goal is to clarify to readers interested in building ontologies from scratch, the activities they should perform and in which order, as well as the set of techniques to be used in each phase of the methodology.

本文并不打算把本体这门手艺完全转变成工程，也不打算穷举这个领域已发表的全部工作。它的目标是向那些有意从零开始建本体的读者说清楚：**他们应该执行哪些活动、按什么顺序执行，以及方法论每个阶段该用哪些技术。**

This paper only presents a set of activities that conform the ontology development process, a life cycle to build ontologies based in evolving prototypes, and METHONTOLOGY, a well-structured methodology used to build ontologies from scratch. This paper gathers the experience of the authors on building an ontology in the domain of chemicals.

本文只呈现三样东西：构成本体开发过程的一组活动；一条基于演进式原型的本体构建生命周期；以及 METHONTOLOGY——一套结构良好的、用于从零开始建本体的方法论。本文汇集了作者在化学品领域构建一个本体的经验。

一、引言：为什么它现在还是手艺

Until now, a big quantity of ontologies have been developed by different groups, under different approaches, and using different methods and techniques. However a few works have been published about how to proceed, showing the practices, design criteria, activities, methodologies, and tools used to build them.

到目前为止，大量本体已经由不同的团队、在不同的路径下、用不同的方法和技术开发出来了。然而，**关于「该怎么进行」的工作却发表得很少**——那些实践、设计判据、活动、方法论和构建工具。

The consequence is clear, the absences of standardized activities, life cycles, and systematic methodologies as well as a set of well-defined design criteria, techniques and tools make the ontologies development a craft rather than an engineering activity.

后果很清楚：**缺少标准化的活动、生命周期、系统化方法论，以及一套定义清楚的设计判据、技术和工具，使得本体开发成了一门手艺，而不是一项工程活动。**

So, the art will become engineering when there exist a definition and standardization of a life cycle that goes from requirements definition to maintenance of the finished product, as well as methodologies and techniques that drive their development.

因此，当存在一条从需求定义一直到成品维护的、被定义和标准化了的生命周期，以及驱动其开发的方法论与技术时，这门手艺才会变成工程。

二、本体与专家系统的本质区别

One of the main problems that Knowledge Engineers (KEs) have when building expert systems is the difficulty of getting a set of requirements for the system. Requirements specify how the system will behave. Since experts are not usually able to describe in a concrete and complete way how they behave in the application domain, it is hard for KEs to specify the future behavior of the Knowledge-Based Systems (KBS).

知识工程师在构建专家系统时的主要难题之一，是难以拿到系统的一组需求。需求要说清系统将怎么表现。而由于专家通常无法具体完整地描述自己在这个应用领域里是怎么做的，知识工程师很难说清知识库系统未来的行为。

So, KBS are usually built incrementally using evolving prototypes in which the deficiencies of the final product of each cycle can be used as the specification of the next prototype.

因此，知识库系统通常用演进式原型增量地构建——每一轮的成品有什么不足，就用来当作下一轮原型的规格说明。

In ontologies do not occur the same. Ontologies are built to be reused or shared anytime, anywhere, and independently of the behavior and domain of the application that uses them. So, ontologists should be able to specify, at least partially, a big portion of the needed vocabulary that the ontology will cover for a given domain.

本体不是这样。 本体被建出来是为了随时随地被复用或共享，并且独立于使用它的那个应用的行为与领域。所以本体构建者应当能够——至少部分地——说清这个本体将为给定领域覆盖的那一大部分所需词汇。

This is the main difference between ontologies and KBS development processes. Consequently, methodologies used to build KBSs can not completely be applied to build ontologies.

这就是本体开发过程与知识库系统开发过程的主要差别。因此，用来建知识库系统的方法论不能完整地套用到建本体上。

三、本体开发过程：先列活动，不定顺序

The ontology development process refers to what activities you need to carry out when building your ontologies. However, the ontology development process does not imply an order of execution of such activities. Its goal is to identify the list of activities to be completed.

本体开发过程指的是你在构建本体时需要执行哪些活动。但要注意，本体开发过程并不暗含这些活动的执行顺序。它的目标是识别出需要完成的活动清单。

Before building your ontology, you should plan the main tasks to be done, how they will be arranged, how much time you need to perform them and with which resources (people, software and hardware).

在构建本体之前，你应当规划要做的主要任务、它们怎么安排、需要多少时间、以及用哪些资源（人、软件、硬件）。

As you do not plan a trip without knowing your destination (your goal), you should not start the development of your ontology without knowing its purpose and scope. So, try to answer the questions: why this ontology is being built and what are its intended uses and end-users. Then, specify or write the answers in an ontology requirements specification document.

就像你不会在不知道目的地的情况下规划一次旅行，你也不该在不知道本体的目的与范围的情况下开始开发它。所以要试着回答这些问题：为什么要建这个本体，它的预期用途是什么，最终用户是谁。然后把答案写进一份**本体需求规格说明文档**。

论文这里给了一个对照：企业建模领域有两个独立的本体，Uschold 的 Enterprise 本体和 Gruninger 的 TOVE 本体。**两者的产出几乎相同（一组术语），但写需求规格说明文档的形式化程度不同**——Enterprise 本体用自然语言写，TOVE 本体用一组**能力问题**（competence questions）写。

四、METHONTOLOGY 的六个阶段

4.1 规格说明

The goal of the specification phase is to produce either an informal, semi-formal or formal ontology specification document written in natural language, using a set of intermediate representations or using competency questions, respectively.

规格说明阶段的目标是产出一份本体规格说明文档，形式可以是非形式的、半形式的或形式的——分别对应用自然语言写、用一组中间表示写、或用能力问题写。

METHONTOLOGY 要求至少包含这三项：

a) The purpose of the ontology, including its intended uses, scenarios of use, end-users, etc.

本体的目的，包括它的预期用途、使用场景、最终用户等。

b) Level of formality of the implemented ontology, depending on the formality that will be use to codify the terms and their meaning.

所实现本体的形式化程度，取决于用什么形式来编码术语及其含义。（Uschold 把形式化程度分成一个区间：高度非形式、半非形式、半形式、严格形式——取决于术语与含义是用接近自然语言还是接近严格形式语言来编码的。）

c) Scope, which includes the set of terms to be represented, its characteristics and granularity.

范围，包括要表示的术语集合、它们的特征与粒度。

关于「中间开花」为什么更好：

The main advantage of the middle-out approach is that it allows you to identify the primary concepts of the ontology you are starting on. After reaching agreement on such terms and their definition you can move on to specialize or generalize them, only if they are necessary. As a result, the terms that you use are more stable, and less re-work and overall effort are required.

中间开花法的主要好处是，**它让你先识别出这个本体的主要概念**。在这些术语及其定义上达成一致之后，**只在必要时**再去特化或泛化它们。结果是：**你用的术语更稳定，需要的返工与总体工作量更少**。

规格说明文档必须满足的三条性质：

Since you can not prove the total completeness of your ontology specification document (any time, anywhere, someone may find a new relevant term to be included), you must guarantee ... the following properties:

既然你无法证明本体规格说明文档的完全完备性（**任何时间、任何地点，都可能有人发现一个该被收进来的新相关术语**），你就必须保证以下性质：

- **Concision**, that is, each and every term is relevant, and there are no duplicated or irrelevant terms. **简洁性**：每一个术语都是相关的，没有重复或不相关的术语。
- **Partial completeness**, which is related with the coverage of the terms, the stopover problem and level of granularity of each and every term. **部分完备性**：与术语的覆盖面、「该停在哪一层」这个问题、以及每一个术语的粒度有关。
- **Consistency**, which refers to all terms and their meanings making sense in the domain. **一致性**：所有术语及其含义在这个领域里都讲得通。

4.2 知识获取

It is important to bear in mind that knowledge acquisition is an independent activity in the ontology development process. However, it is coincident with other activities. As we told before, most of the acquisition is done simultaneously with the requirements specification phase, and decreases as the ontology development process moves forward.

要记住一点：知识获取在本体开发过程中是一项**独立的活动**，但它与其他活动重叠。**大部分获取工作和需求规格说明阶段同时进行，并随着开发过程推进而递减**。

Experts, books, handbooks, figures, tables and even other ontologies are sources of knowledge from which the knowledge can be elucidated using in conjunction techniques such as: brainstorming, interviews, formal and informal analysis of texts, and knowledge acquisition tools.

专家、书籍、手册、图、表，**乃至其他本体**，都是知识来源。可以配合使用这些技术把知识抽取出来：头脑风暴、访谈、文本的形式与非形式分析、以及知识获取工具。

论文给了他们在化学品本体上实际用的技术顺序：**非结构化专家访谈**（做需求规格说明的初稿）、**非形式文本分析**（研究书籍手册里的主要概念，用来填充概念化阶段的中间表示）、**形式文本分析**（先识别要检测的结构——定义、断言等——以及每种结构贡献的知识类型）。

4.3 概念化

In this activity, you will structure the domain knowledge in a conceptual model that describes the problem and its solution in terms of the domain vocabulary identified in the ontology specification activity.

在这项活动里，你要把领域知识组织成一个概念模型，**用规格说明阶段识别出的领域词汇**来描述问题及其解法。

The first thing to do is to build a complete Glossary of Terms (GT). Terms include concepts, instances, verbs and properties. Note that you do not start from scratch when you develop your GT. If you made a good specification document, many terms will have been identified in that document.

第一件事是建一份完整的术语表。 术语包括概念、实例、动词和属性。注意：**你不是从零开始做这份术语表——如果你的规格说明文档做得好，很多术语已经在那份文档里被识别出来了。**

Once you have almost completed the GT, you must group terms as concepts and verbs. ... for each set of related concepts and related verbs, a concepts classification tree and a verbs diagram is built. After they have been built, you can split your ontology development process into different, but related, teams.

术语表基本完成之后，要把术语分成**概念**和**动词**。……对每一组相关概念和相关动词，分别建一棵概念分类树和一张动词图。**建好之后，你就可以把本体开发过程拆给不同但相关的团队。**

概念用这些中间表示来描述：**数据字典**（收集全部有用与潜在有用的领域概念、它们的含义、属性、实例）、**实例属性表**、**类属性表**（描述概念本身而不是它的实例）、**常量表**、**实例表**、**属性分类树**（图形化展示属性与常量在根属性推断序列中的关系，以及要执行的公式或规则序列）。

动词代表领域中的动作，用**动词字典**（以声明方式表达动词含义）、**条件表**（执行一个动作前要满足的条件，或执行后要保证的条件）来描述。**公式表和规则表两支都用。**

In sum, METHONTOLOGY produces in this phase a conceptual model expressed as a set of well-defined deliverables that will allow to the final users: (a) to ascertain whether or not an ontology is useful and usable for a given application without inspecting its source code; and (b) to compare the scope and completeness of several ontologies.

总之，METHONTOLOGY 在这个阶段产出一个由[一组明确定义的交付物](#)表达的概念模型，它让最终用户能够：**(a) 在不检查源码的情况下判断这个本体对某个给定应用是否有用可用；(b) 比较几个本体的范围与完备性、以及它们的可复用性和可共享性。**

4.4 集成

With the goal of speeding up the construction of your ontology, you might consider reuse of definitions already built into other ontologies instead of starting from scratch.

为了加快本体的构建，你可以考虑[复用已经建在其他本体里的定义](#)，而不是从零开始。

论文提出两步：

1. **先检视元本体**（比如 Cyc、Ontolingua 里的），挑出最契合你的概念化的那些。**目的是保证新定义与被复用的定义建立在同一组基础术语之上。**
2. 无论是否复用现成元本体，**下一步是找出哪些本体库提供的术语定义，其语义与实现和你概念化里识别出的术语是一致的。**选定之后，如果那些术语所基于的元本体与你用的不同，**你要检查是否存在翻译器能把定义转换到你的目标语言。**

Sometimes it might occur that a term in your conceptualization (e.g., centimeter), that should be included in a given ontology (e.g., Standard Units in Ontolingua) is not provided by the ontology server. In this case, you should justify the need to include the missed definitions as well as the benefits of such inclusion to the ontology maintainer.

有时会遇到这种情况：你概念化里的某个术语（比如「厘米」）本该被包含在某个既有本体里（比如 Ontolingua 的标准单位本体），[但本体服务器没有提供它](#)。这时你应当向那个本体的维护者论证为什么需要补上这些缺失的定义、以及补上的好处。

产物是一份[集成文档](#)，逐个术语记下：概念模型里的术语名、你将从哪个本体取它的定义、那个定义在源本体里的名字及其参数。

4.5 实现

The result of this phase is the ontology codified in a formal language such as: CLASSIC, BACK, LOOM, Ontolingua, Prolog, C++ or in your favorite language.

这个阶段的产物是用形式语言编码的本体——CLASSIC、BACK、LOOM、Ontolingua、Prolog、C++ 或你偏好的语言。

Any ontology development environment should provide, at least: a lexical and syntactic analyzer to guarantee the absence of lexical and syntactic errors; translators, to guarantee the portability of the definitions into other target languages; an editor; a browser; a searcher; [evaluators, to detect incompleteness, inconsistencies and redundant knowledge](#); an automatic maintainer, to manage the inclusion, removal or modification of existing definitions.

任何本体开发环境至少应提供：[词法与语法分析器](#)（保证没有词法与语法错误）；[翻译器](#)（保证定义可移植到其他目标语言）；[编辑器](#)（增删改定义）；[浏览器](#)（检视本体库及其定义）；[检索器](#)（找最合适的定义）；[评估器](#)（检测不完整、不一致和冗余的知识）；[自动维护器](#)（管理既有定义的加入、移除、修改）。

4.6 评估

评估的含义是对本体、相关软件环境与文档做一次[技术判断](#)，判断它们相对于一个参照框架的完成度。论文引用了作者团队自己的另一篇框架论文，并强调[评估贯穿每个阶段，不是最后一道工序](#)。

参考链接

- [AAAI Technical Report SS-97-06](#) — 本文所属的那次 AAI 春季研讨会论文集
- Uschold & Gruninger 1996 — 论文里对照的另两套方法论（Enterprise 本体用自然语言写规格说明，TOVE 本体用能力问题写），以及「中间开花优于自上而下与自下而上」这个论证的出处